Politechnika Wrocławska

Wydział Informatyki
i Telekomunikacji

Algorytmy i Złożoność obliczeniowa: projekt

Algorytmy sortowania i ich złożoności obliczeniowe

Miniewski Dawid, 284556

Prowadzący:
dr inż. Z. Buchalski

Wrocław 2026

Spis treści

1 Cel projektu **3**

2 Podstawy teoretyczne algorytmów sortowania **3**
2.1 Heap Sort (Sortowanie przez kopcowanie) 3
2.2 Shell Sort (Sortowanie Shella) 3
2.3 Quick Sort (Sortowanie szybkie) 3
2.4 Hybrid Sort (Sortowanie hybrydowe) 4

3 Plan i metodologia eksperymentu **4**
3.1 Środowisko sprzętowe i programowe 4
3.2 Założenia pomiarowe 4
3.3 Sposób pomiaru czasu i zabezpieczenia 5

4 Omówienie przebiegu eksperymentów i wyniki **5**
4.1 Analiza wyników 5

5 Prezentacja uzyskanych wyników **5**
5.1 Algorytm Hybrid Sort (połączenie Quick Sort i Insertion Sort) 5
5.2 Algorytm Quick Sort (wpływ wyboru pivota) 8
5.3 Algorytm Heap Sort 12
5.4 Algorytm Shell Sort (porównanie sekwencji odstępów) 13
5.5 Zbiorcze zestawienie wyników 15

6 Podsumowanie i wnioski **17**

7 Literatura **19**

1 Cel projektu

Głównym celem projektu jest empiryczna analiza i porównanie wydajności wybranych algorytmów sortowania (Heap Sort, Shell Sort, Quick Sort oraz Hybrid Sort) w zależności od rozmiaru struktury danych, początkowego stopnia uporządkowania elementów oraz typu sortowanych danych (*int* vs *float*).

Eksperyment ma na celu zbadanie rzeczywistego czasu wykonywania algorytmów i zweryfikowanie teoretycznych założeń dotyczących ich złożoności obliczeniowej, ze szczególnym uwzględnieniem przypadków średnich i pesymistycznych. Dodatkowo projekt zakłada zbadanie wpływu optymalizacji, takich jak wybór elementu osiowego (ang. *pivot*) w sortowaniu szybkim oraz zastosowanie optymalnego progu odcięcia (ang. *threshold*) w autorskim algorytmie hybrydowym.

2 Podstawy teoretyczne algorytmów sortowania

2.1 Heap Sort (Sortowanie przez kopcowanie)

Algorytm sortowania przez kopcowanie opiera się na strukturze danych zwanej kopcem binarnym (ang. *binary heap*). Działanie algorytmu dzieli się na dwie fazy: budowę kopca typu *max-heap* z nieuporządkowanej tablicy oraz właściwe sortowanie, polegające na wielokrotnym pobieraniu elementu maksymalnego (korzenia) i przywracaniu własności kopca [1].

- **Złożoność czasowa (przypadek średni):** $O(N \log N)$
- **Złożoność czasowa (przypadek najgorszy):** $O(N \log N)$

Mimo optymalnej i pewnej złożoności asymptotycznej w każdym przypadku, algorytm ten w praktyce często ulega sortowaniu szybkiemu z powodu bardzo słabej lokalności odwołań do pamięci (ang. *cache locality*), co zostało zweryfikowane w niniejszym eksperymencie.

2.2 Shell Sort (Sortowanie Shella)

Sortowanie Shella jest uogólnieniem sortowania przez wstawianie (ang. *Insertion Sort*), które dopuszcza wymianę elementów oddalonych od siebie o pewien odstęp (ang. *gap*). Z każdą iteracją odstęp ten jest zmniejszany, aż do wartości 1, co sprowadza algorytm w ostatniej fazie do klasycznego wstawiania [1]. Wydajność tego algorytmu jest determinowana przez matematyczną formułę generującą sekwencję odstępów.

- **Złożoność czasowa (przypadek średni):** Zależna od sekwencji (często szacowana w przedziale od $O(N^{1.25})$ do $O(N^{1.5})$).
- **Złożoność czasowa (przypadek najgorszy):** Dla oryginalnej sekwencji zaproponowanej przez Shella wynosi $O(N^2)$, jednak dla ulepszonych sekwencji (w tym badanej sekwencji Franka i Lazarusa) złożoność ulega znacznej poprawie i ograniczeniu z góry.

2.3 Quick Sort (Sortowanie szybkie)

Sortowanie szybkie to klasyczny algorytm działający w oparciu o paradygmat „dziel i zwyciężaj". Polega na wyborze elementu osiowego (pivota) i podziale tablicy na dwie podtablice: zawierającą elementy mniejsze bądź równe pivotowi oraz zawierającą elementy większe [1]. Rekurencja jest kontynuowana dla obu części aż do uzyskania tablic jednoelementowych.

- **Złożoność czasowa (przypadek średni):** $O(N \log N)$
- **Złożoność czasowa (przypadek najgorszy):** $O(N^2)$

Przypadek pesymistyczny występuje, gdy podziały są skrajnie nierównomierne (np. przy wyborze skrajnego elementu jako pivota w tablicy już posortowanej, co prowadzi do bardzo głębokiej rekurencji).

2.4 Hybrid Sort (Sortowanie hybrydowe)

Zaprojektowany algorytm hybrydowy łączy zalety doskonałej złożoności asymptotycznej Quick Sorta z niskim narzutem operacyjnym sortowania przez wstawianie dla małych zestawów danych. Mechanizm zakłada, że gdy rozmiar podtablicy spada poniżej predefiniowanego progu odcięcia (w badaniach: 10, 25, 100), rekurencja ulega przerwaniu, a podtablica jest sortowana lokalnie z wykorzystaniem Insertion Sort.

- **Złożoność czasowa (przypadek średni):** $O(N \log N)$
- **Złożoność czasowa (przypadek najgorszy):** $O(N^2)$ (z uwagi na oparcie mechanizmu podziału na klasycznym wariancie Quick Sort).

Podejście to minimalizuje obciążenie stosu wywołań oraz optymalizuje wykorzystanie najszybszej pamięci podręcznej procesora.

3 Plan i metodologia eksperymentu

3.1 Środowisko sprzętowe i programowe

Aby zapewnić wiarygodność i powtarzalność wyników, testy zostały przeprowadzone w izolowanym środowisku przy zminimalizowanym wpływie procesów w tle. Specyfikacja platformy testowej:

- **Procesor:** Intel Core i5-13600KF (Taktowanie bazowe: 3.50 GHz, Cache L1: 1.2 MB, L2: 20.0 MB, L3: 24.0 MB)
- **Pamięć RAM:** 32.0 GB, taktowanie 3733 MT/s
- **System operacyjny:** Windows 11 (64-bit)
- **Kompilator:** g++.exe (GCC) 11.2.0, flagi: `g++ *.cpp -O3 -o program.exe`

3.2 Założenia pomiarowe

Eksperyment przeprowadzono dla dwóch typów danych: całkowitych (`int`) oraz zmiennoprzecinkowych (`float`). Aby zaobserwować rzeczywiste tempo wzrostu czasu wykonania algorytmów, wybrano następujące rozmiary tablic (reprezentowane przez zmienną N): 50 000, 100 000, 250 000, 500 000, 1 000 000, 2 500 000 oraz 5 000 000 elementów.

Dla każdego algorytmu, każdego rozmiaru tablicy oraz każdego z badanych przypadków początkowego uporządkowania symulacja generowała dane według następujących scenariuszy:

1. **Dane losowe:** Elementy wygenerowane z rozkładu jednostajnego za pomocą standardowych funkcji pseudolosowych.

2. **Dane posortowane (rosnąco):** Stanowią przypadek optymistyczny dla Insertion Sorta, a pesymistyczny m.in. dla naiwnego Quick Sorta z lewym pivotem.

3. **Dane posortowane odwrotnie (malejąco):** Klasyczny test weryfikujący stabilność i zachowanie algorytmów w skrajnie niekorzystnych warunkach.

4. **Dane częściowo posortowane (33% oraz 66%):** Symulują rzeczywiste zbiory danych, które często mają już pewien stopień uporządkowania.

3.3 Sposób pomiaru czasu i zabezpieczenia

Czas sortowania mierzono z wykorzystaniem precyzyjnej biblioteki `<chrono>` (konkretnie) `high_resolution_clock`). Aby wyeliminować anomalie i szumy wynikające z działania systemu operacyjnego, każdy wariant testowy był powtarzany **100 razy**, a końcowym wynikiem prezentowanym w sprawozdaniu jest średnia arytmetyczna tych prób wyrażona w milisekundach (ms).

Dodatkowo, w celu uniknięcia krytycznych błędów naruszenia pamięci (tzw. *Stack Overflow*) w pesymistycznych przypadkach algorytmu Quick Sort na posortowanych tablicach, zaimplementowano własny mechanizm obronny. Algorytmy rekurencyjne monitorowały aktualną głębokość wywołań stosu, rzucając wyjątek (`std::runtime_error`) po przekroczeniu ustalonego limitu, co pozwalało na bezpieczne przerwanie pojedynczego testu bez przerywania całej zautomatyzowanej symulacji.

4 Omówienie przebiegu eksperymentów i wyniki

4.1 Analiza wyników

Uzyskane dane pomiarowe z agregacji 100 powtórzeń dla każdego wariantu zostały przedstawione na poniższych wykresach. Zgodnie z założeniami projektu, dla każdego wariantu algorytmu i typu danych (INT/Float) sporządzono zbiorczy wykres liniowy ilustrujący czas sortowania w funkcji rozmiaru tablicy (N). Pojedynczy wykres zawiera pięć krzywych, odpowiadających różnym dystrybucjom początkowym danych (losowe, posortowane, odwrotnie posortowane, 33% i 66% posortowania).

W przypadkach, gdy program zgłosił błąd przepełnienia stosu (*Stack Overflow*) - co wystąpiło w wariantach Quick Sort (Lewy/Prawy) dla danych innych niż losowe - testy dla rosnącego N zostały przerwane, co objawia się brakiem punktów pomiarowych na wykresie dla większych rozmiarów tablic.

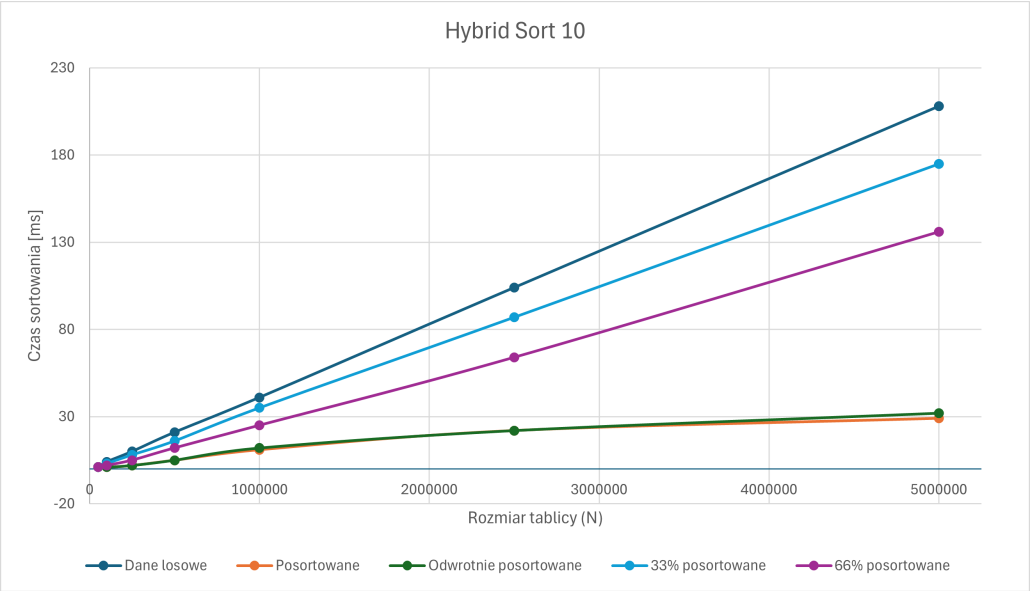
5 Prezentacja uzyskanych wyników

Poniżej przedstawiono wyniki przeprowadzonych symulacji, prezentujące czas sortowania w funkcji rozmiaru tablicy (N). Zgodnie z przyjętą metodologią, dla każdej metody sortowania oraz typu danych (całkowite `int` oraz zmiennoprzecinkowe `float`) wygenerowano osobny wykres, na którym pięć krzywych reprezentuje różny stopień początkowego uporządkowania zbioru.

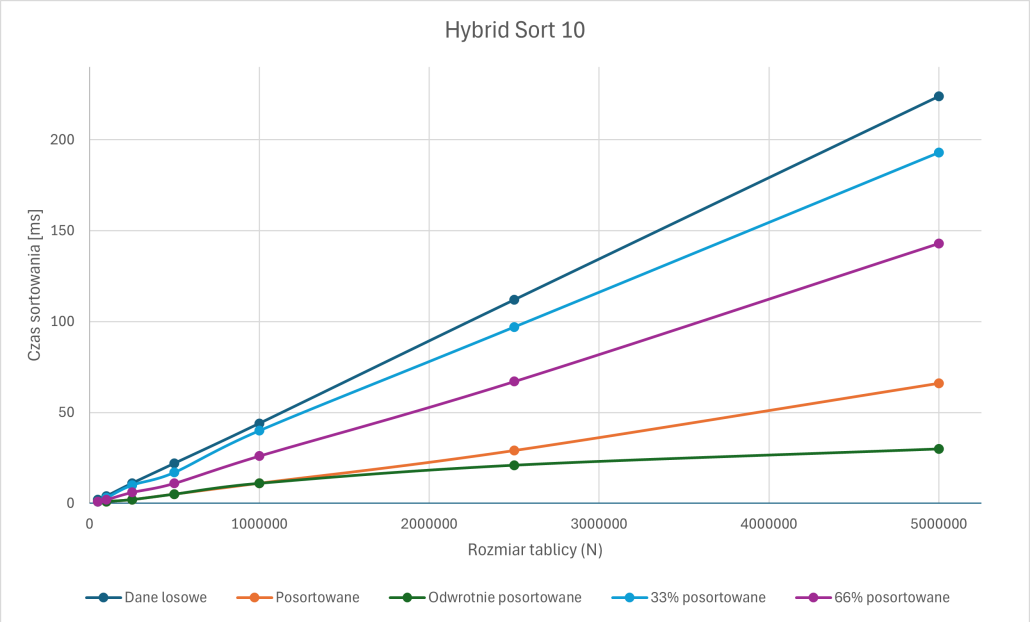
5.1 Algorytm Hybrid Sort (połączenie Quick Sort i Insertion Sort)

Poniższe zestawienia prezentują wydajność autorskiego algorytmu hybrydowego dla trzech badanych progów odcięcia (threshold = 10, 25 oraz 100). Algorytm ten domyślnie korzystał ze

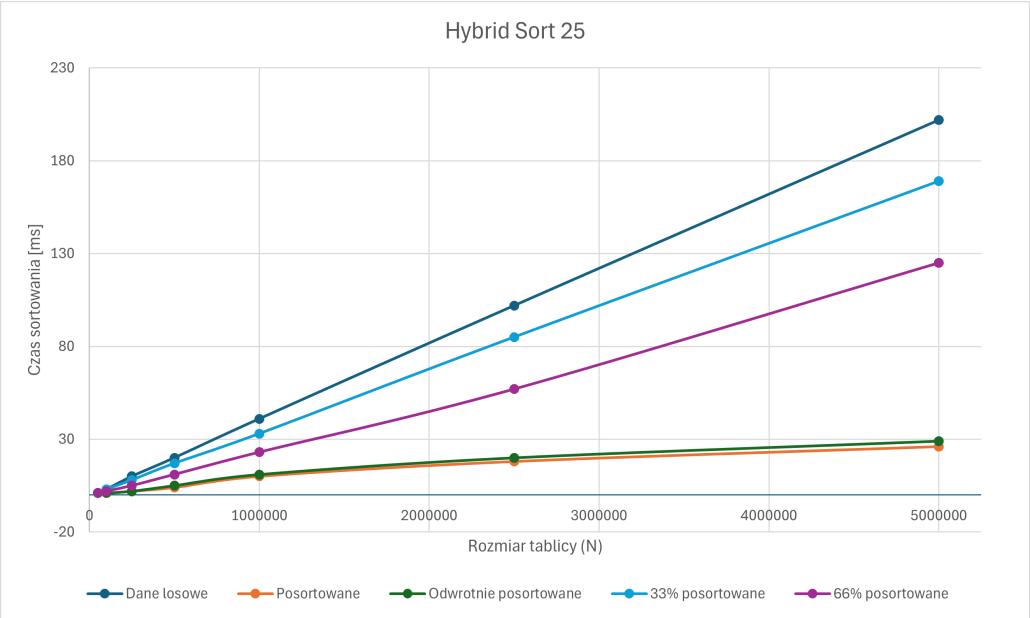
środkowego elementu osiowego. Uwagę zwraca jego znakomite radzenie sobie z danymi ułożonymi odwrotnie oraz wstępnie posortowanymi – w tych przypadkach czas wykonania jest znacznie krótszy niż dla danych losowych, co wynika z idealnego dzielenia tablicy na równe połowy i bardzo płytkiej rekurencji. Zbyt mały próg (10) wykazuje delikatny narzut czasowy względem optymalnego progu (25).



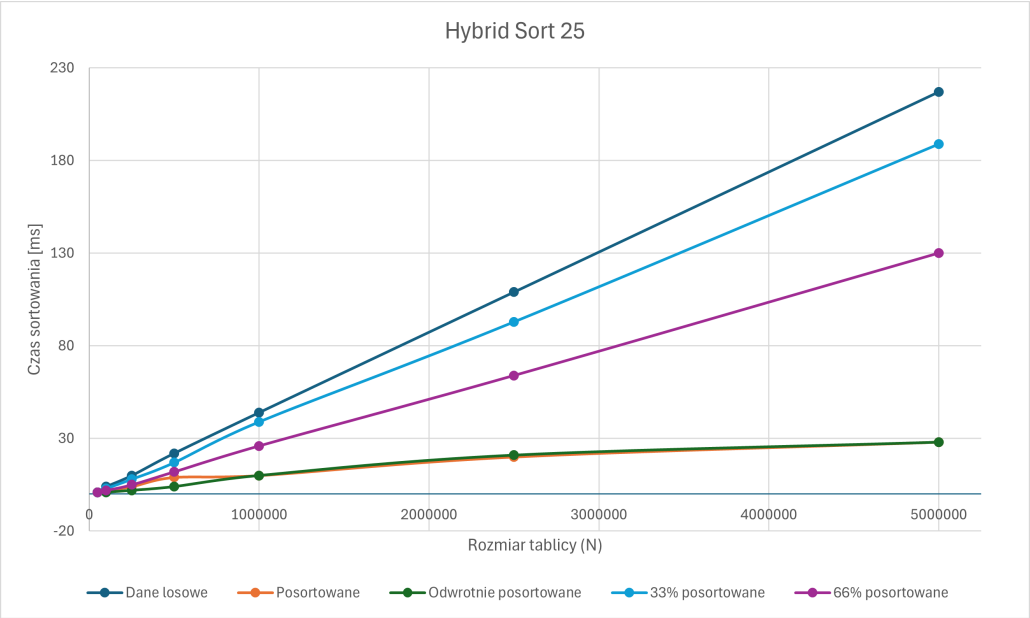
Rysunek 1: Wydajność algorytmu Hybrid Sort (próg 10) dla typu INT



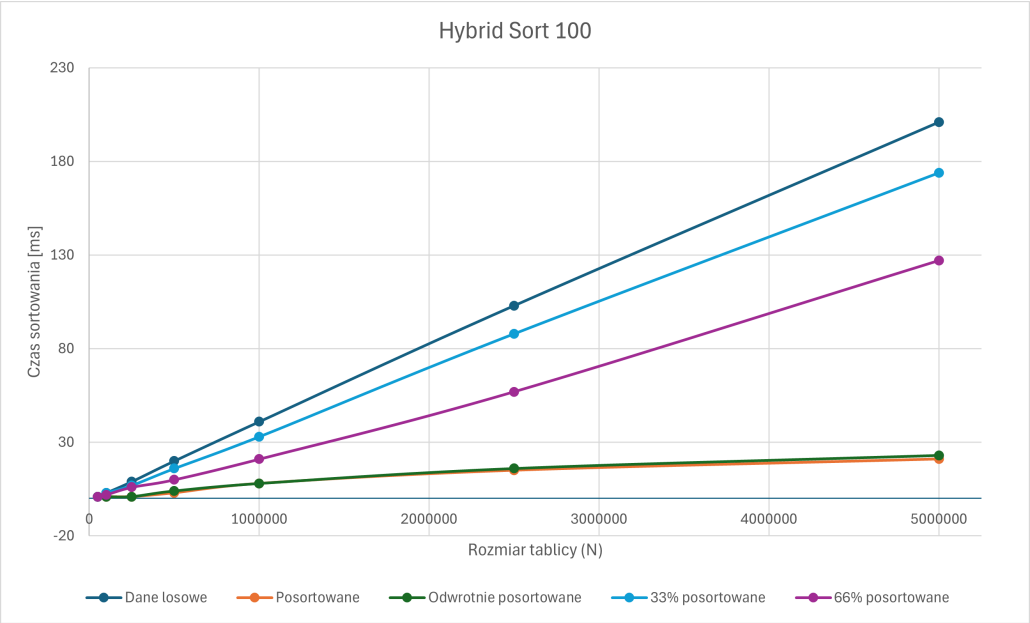
Rysunek 2: Wydajność algorytmu Hybrid Sort (próg 10) dla typu FLOAT



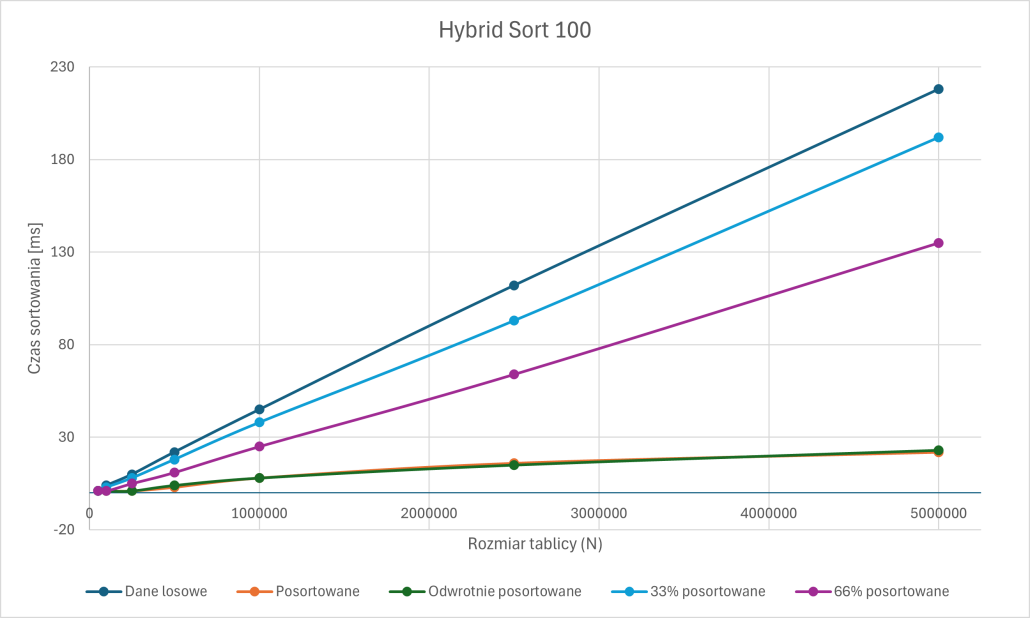
Rysunek 3: Wydajność algorytmu Hybrid Sort (próg 25) dla typu INT



Rysunek 4: Wydajność algorytmu Hybrid Sort (próg 25) dla typu FLOAT



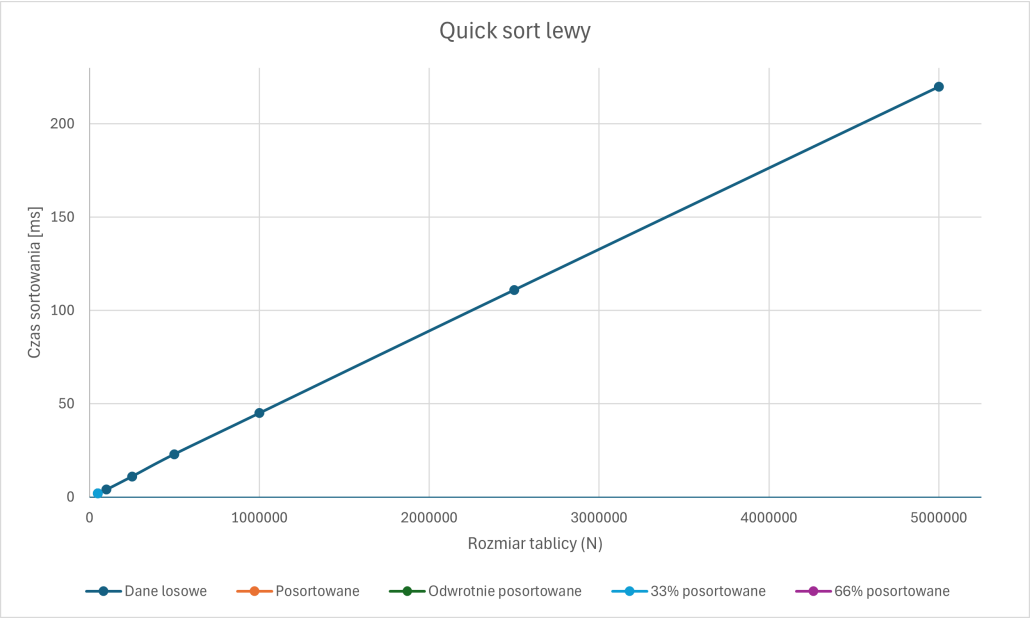
Rysunek 5: Wydajność algorytmu Hybrid Sort (próg 100) dla typu INT



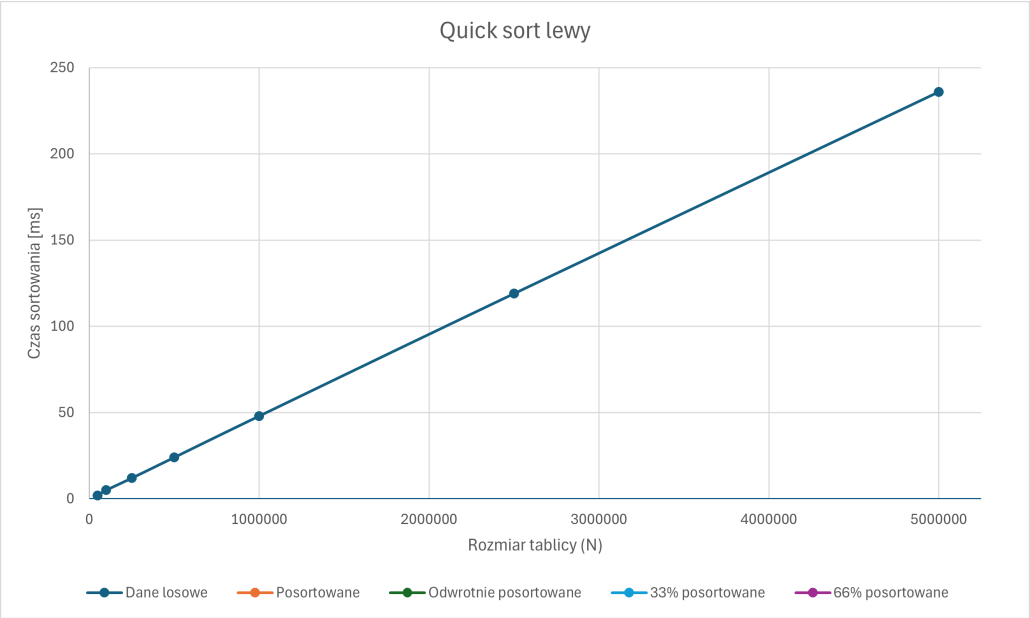
Rysunek 6: Wydajność algorytmu Hybrid Sort (próg 100) dla typu FLOAT

5.2 Algorytm Quick Sort (wpływ wyboru pivota)

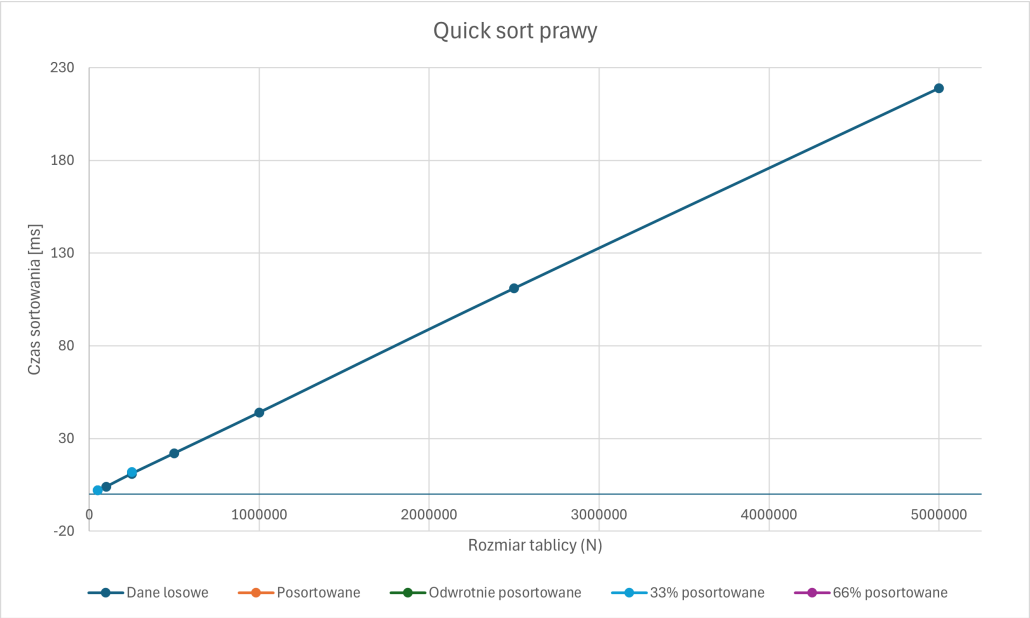
Kolejna sekcja obrazuje zachowanie klasycznego algorytmu Quick Sort. W przypadku wyboru skrajnego elementu osiowego (lewy lub prawy), dla danych częściowo lub całkowicie posortowanych, linie trendu urywają się już po pierwszym pomiarze. Jest to wizualna reprezentacja wystąpienia krytycznego błędu przepełnienia stosu (*Stack Overflow*), wywołanego degradacją złożoności obliczeniowej do $O(N^2)$. Problem ten całkowicie znika przy zastosowaniu pivota środkowego lub losowego. Należy jednak zauważyć, że pivot losowy wprowadza zauważalny narzut czasowy dla danych całkowicie losowych z powodu konieczności ciągłego wywoływania funkcji pseudolosowej.



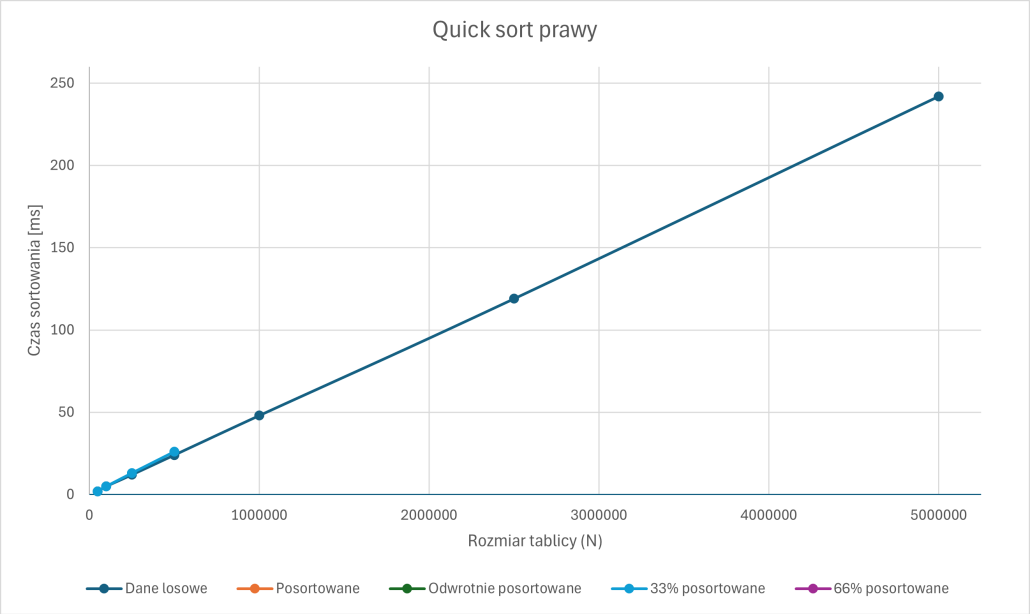
Rysunek 7: Wydajność algorytmu Quick Sort (Lewy) dla typu INT



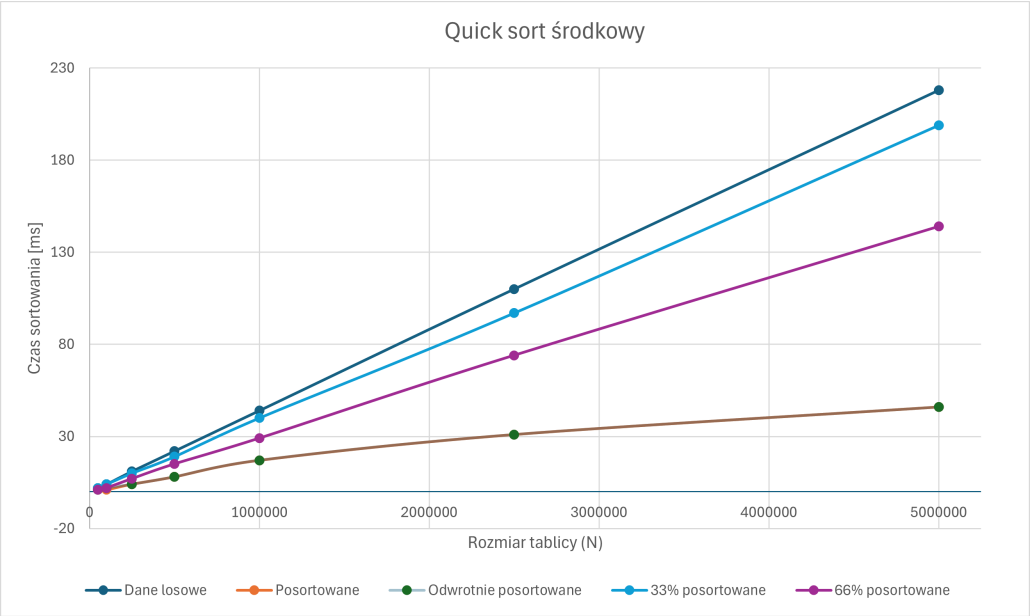
Rysunek 8: Wydajność algorytmu Quick Sort (Lewy) dla typu FLOAT



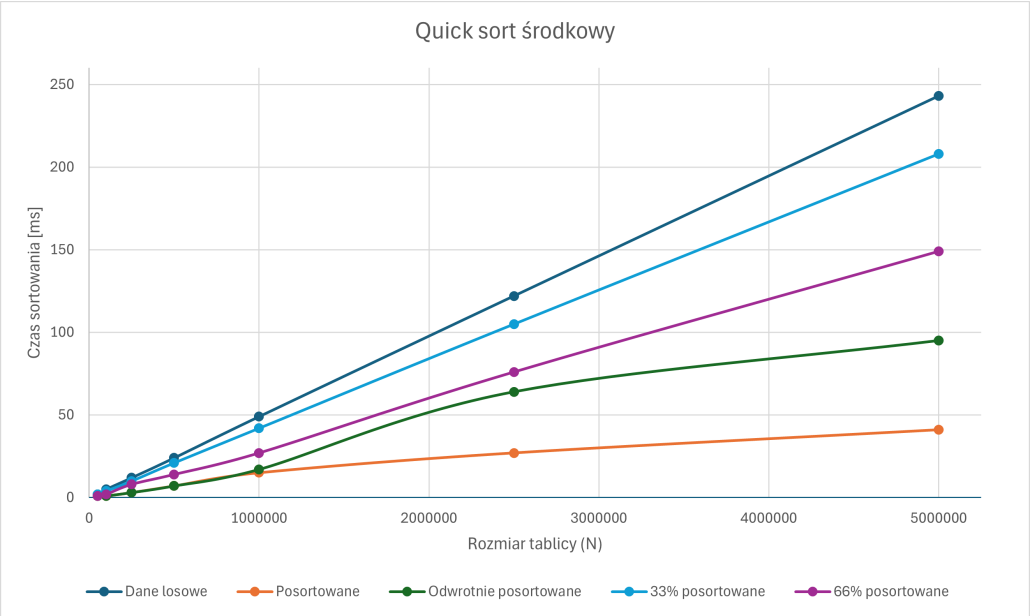
Rysunek 9: Wydajność algorytmu Quick Sort (Prawy) dla typu INT



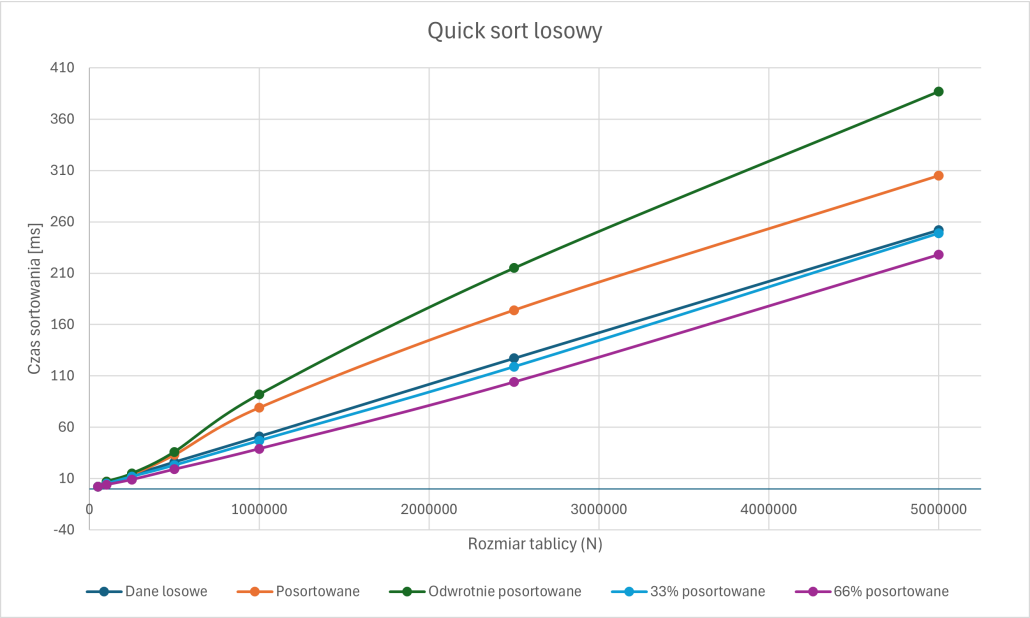
Rysunek 10: Wydajność algorytmu Quick Sort (Prawy) dla typu FLOAT



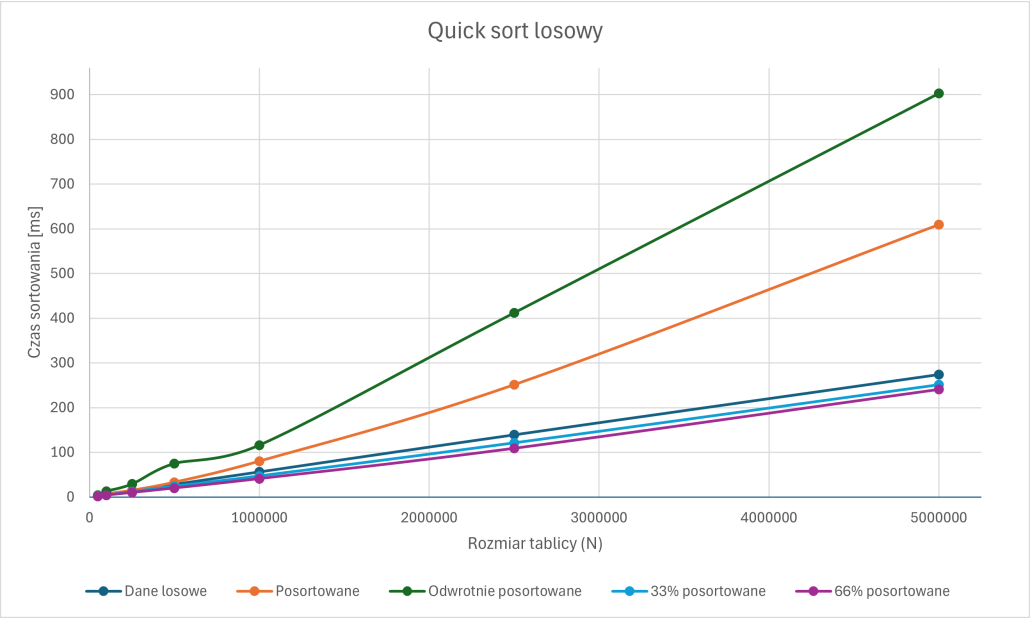
Rysunek 11: Wydajność algorytmu Quick Sort (Środkowy) dla typu INT



Rysunek 12: Wydajność algorytmu Quick Sort (Środkowy) dla typu FLOAT



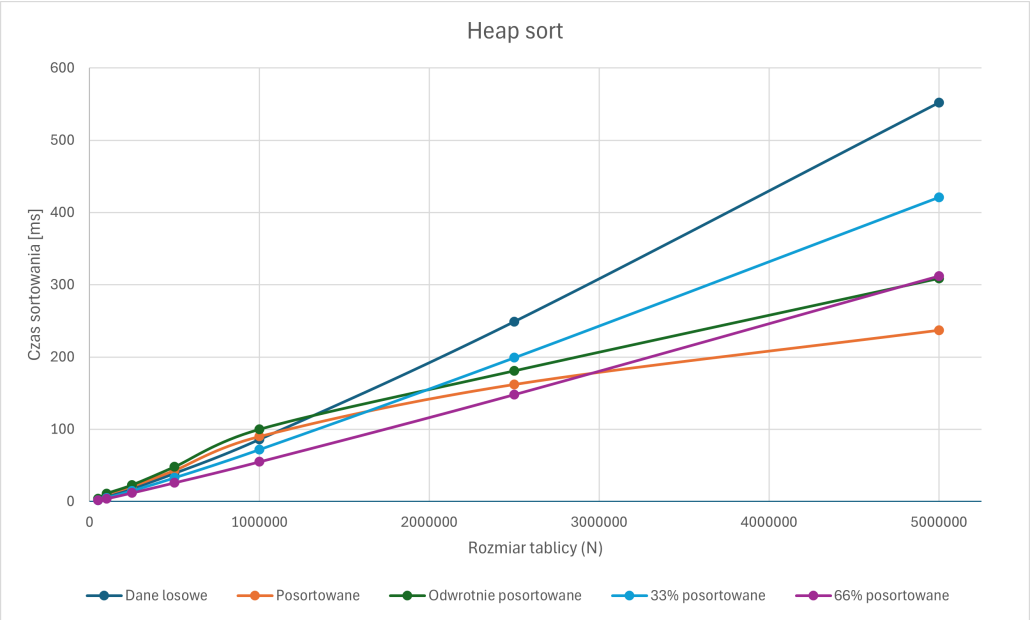
Rysunek 13: Wydajność algorytmu Quick Sort (Losowy) dla typu INT



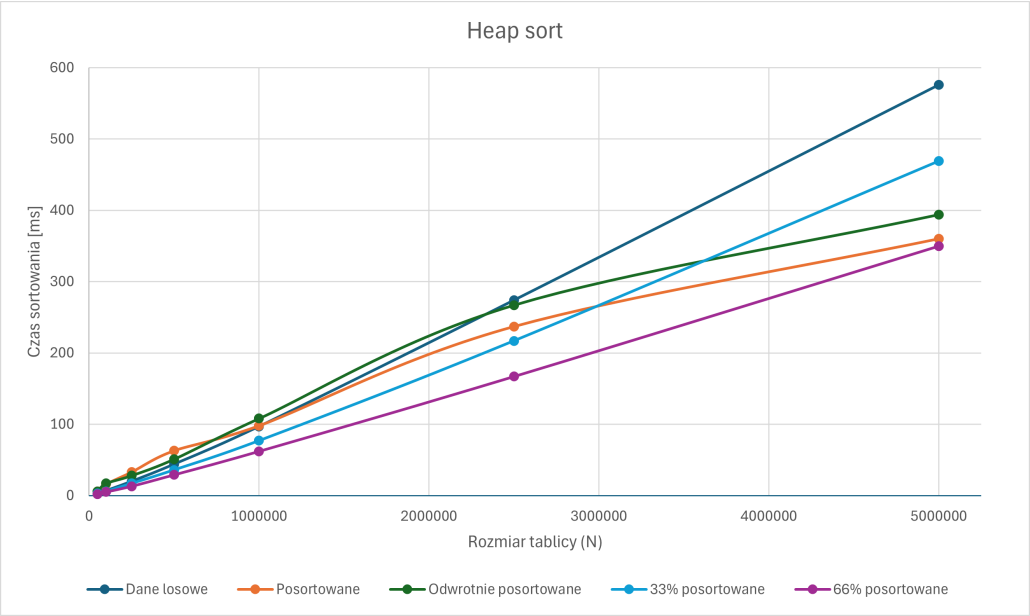
Rysunek 14: Wydajność algorytmu Quick Sort (Losowy) dla typu FLOAT

5.3 Algorytm Heap Sort

Wykresy dla algorytmu Heap Sort stanowią doskonałe potwierdzenie jego stabilnej, teoretycznej złożoności na poziomie $O(N \log N)$. W przeciwieństwie do Quick Sorta czy algorytmów hybrydowych, krzywe dla różnych rozkładów danych układają się w bardzo wąską wiązkę. Dowodzi to, że struktura kopca zawsze wymaga zbliżonej liczby przemieszczeń w pamięci, niezależnie od stopnia początkowego uporządkowania tablicy.



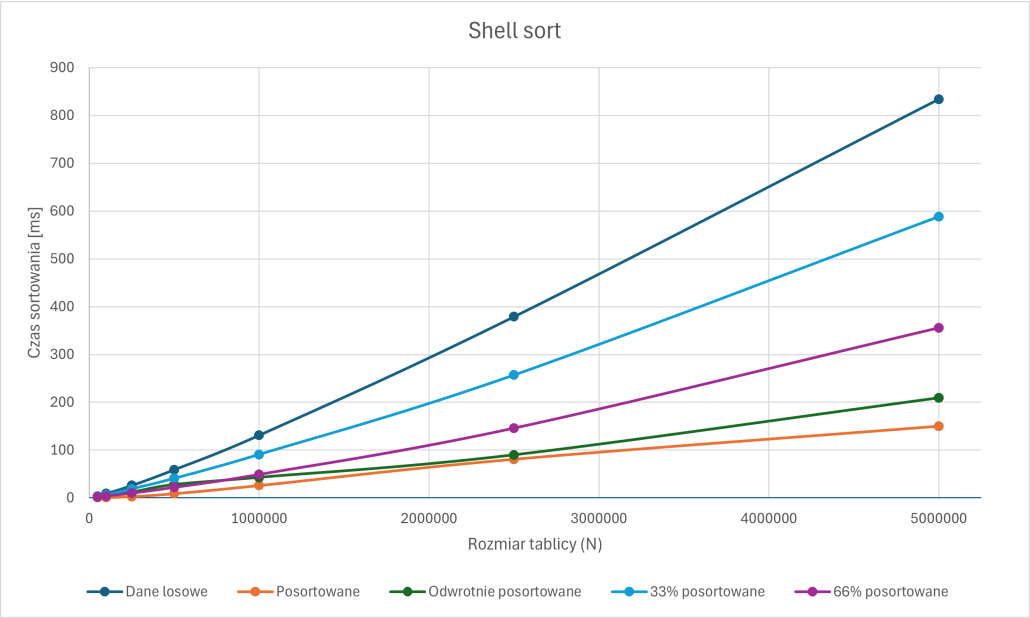
Rysunek 15: Wydajność algorytmu Heap Sort dla typu INT



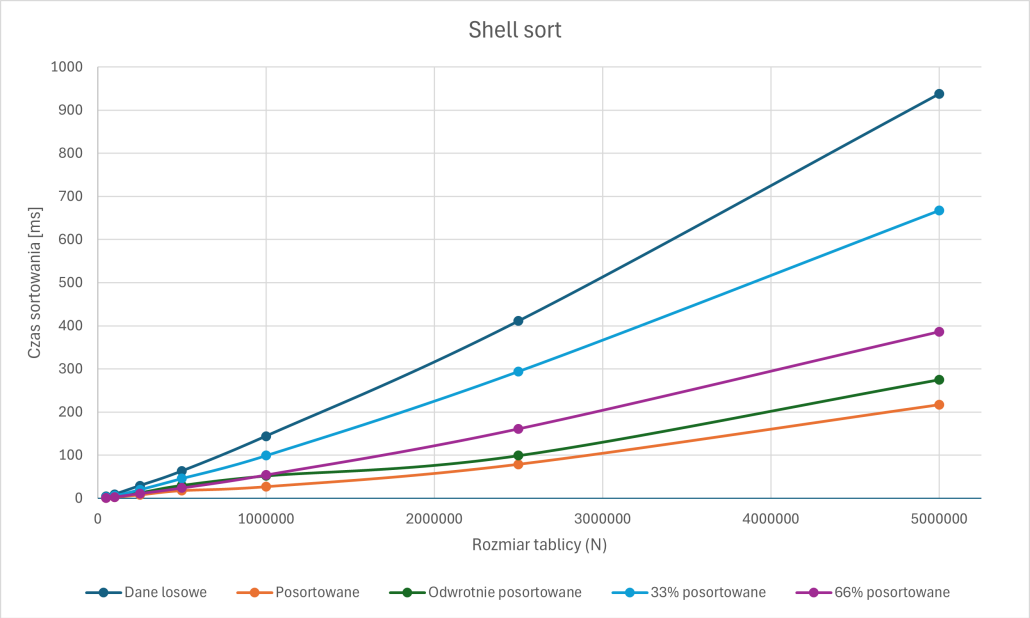
Rysunek 16: Wydajność algorytmu Heap Sort dla typu FLOAT

5.4 Algorytm Shell Sort (porównanie sekwencji odstępów)

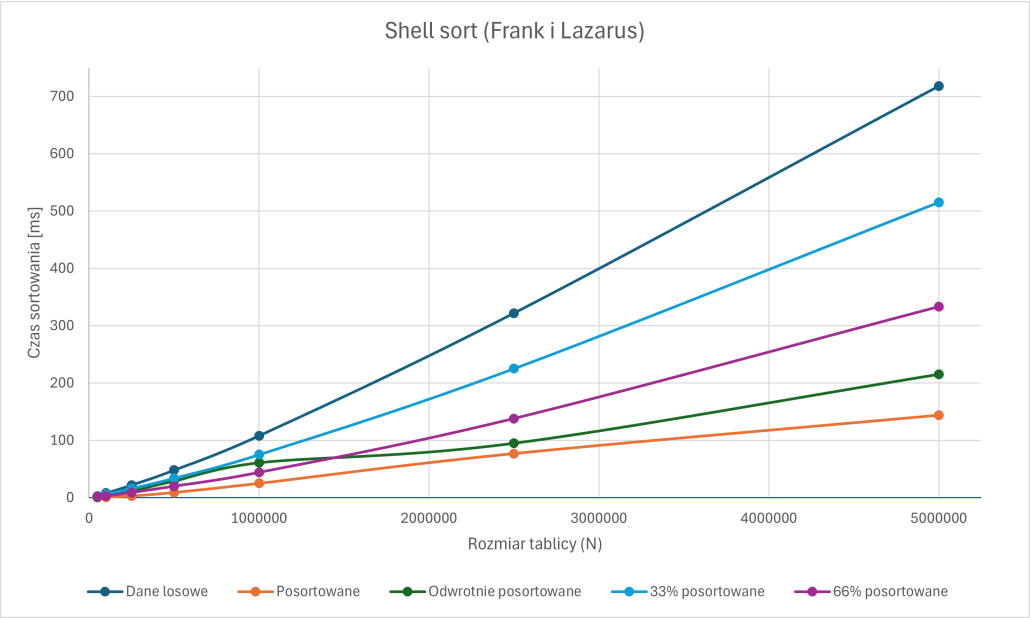
Ostatnie zestawienie ukazuje różnicę w wydajności sortowania Shella w zależności od zastosowanej sekwencji odstępów. Dla mniejszych zestawów danych krzywe niemal się pokrywają. Jednak dla największych badanych rozmiarów ($N \geq 2.5 \times 10^6$), matematycznie zoptymalizowana sekwencja Franka i Lazarusa drastycznie spłaszcza krzywą wzrostu czasu, podczas gdy klasyczna metoda Shella zaczyna zdradzać wyraźne tendencje do złożoności kwadratowej, co jest szczególnie widoczne dla danych całkowicie losowych.



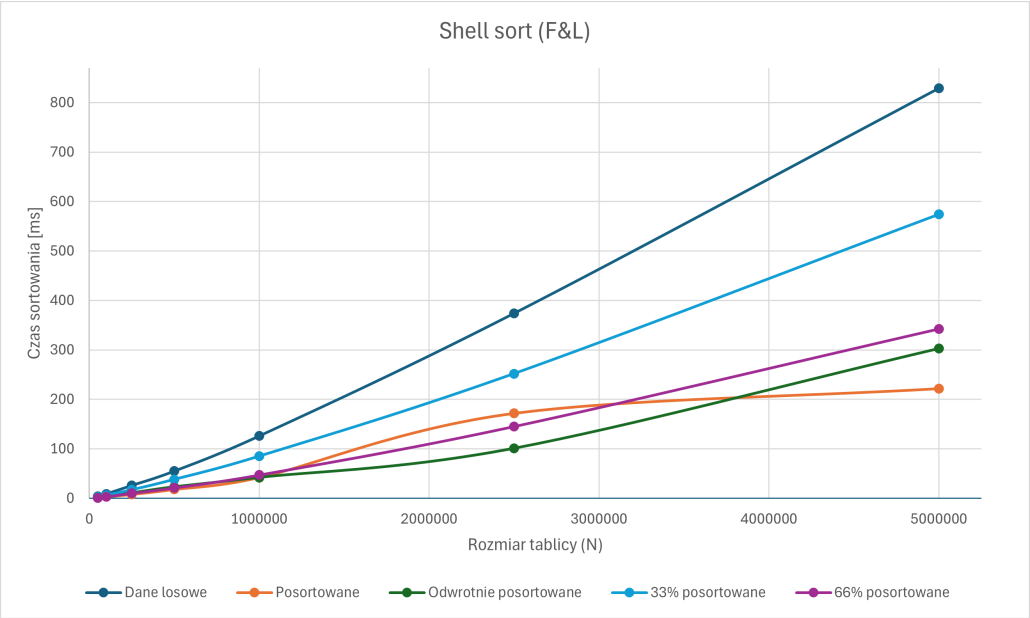
Rysunek 17: Wydajność algorytmu Shell Sort (klasyczny) dla typu INT



Rysunek 18: Wydajność algorytmu Shell Sort (klasyczny) dla typu FLOAT



Rysunek 19: Wydajność algorytmu Shell Sort (Frank & Lazarus) dla typu INT



Rysunek 20: Wydajność algorytmu Shell Sort (Frank & Lazarus) dla typu FLOAT

5.5 Zbiorcze zestawienie wyników

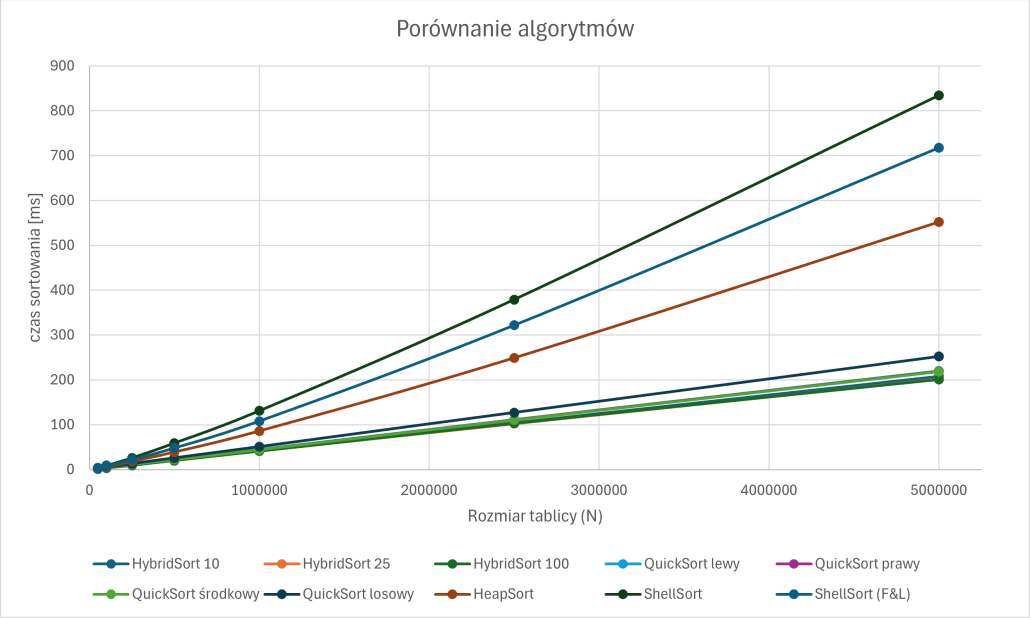
Aby ułatwić bezpośrednie porównanie wydajności zaimplementowanych algorytmów, w Tabeli 1 przedstawiono ostateczne czasy sortowania dla największego badanego rozmiaru struktury danych ($N = 5\,000\,000$ elementów) w wariancie dla liczb całkowitych (`int`).

Tabela ta doskonale uwydatnia przepaść wydajnościową między algorytmami dla poszczególnych rozkładów danych, a także obrazuje awarię klasycznego sortowania szybkiego (oznaczoną jako *S.O.* – Stack Overflow) przy niekorzystnym układzie elementów.

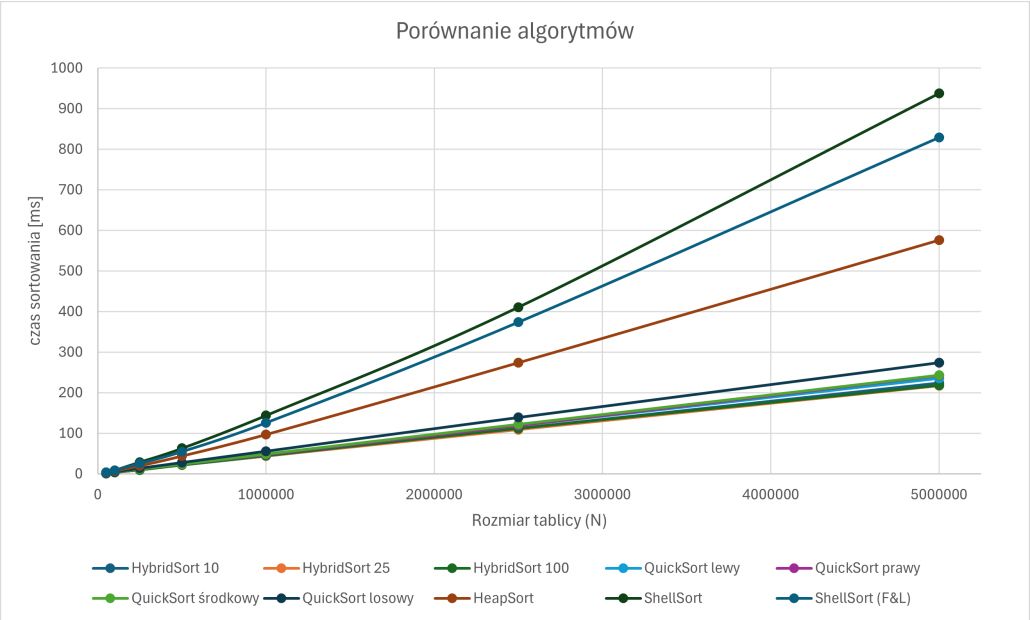
Tabela 1: Zestawienie czasów sortowania [ms] dla tablicy $N = 5\,000\,000$ elementów (Typ INT)

Algorytm	**Losowe**	**Posortowane**	**Odwrotnie**	**33% posort.**	**66% posort.**
Hybrid Sort (25)	**202**	**26**	**29**	**169**	**125**
Quick Sort (Środkowy)	218	46	46	199	144
Quick Sort (Lewy)	220	*S.O.*	*S.O.*	*S.O.*	*S.O.*
Quick Sort (Losowy)	252	305	387	249	228
Heap Sort	552	237	309	421	312
Shell Sort (F&L)	718	144	215	515	333
Shell Sort (Klasyczny)	834	150	209	588	356

Pełne, surowe tabele pomiarowe dla wszystkich badanych rozmiarów N oraz obu typów danych zostały dołączone do elektronicznej wersji projektu na dysku Google.



Rysunek 21: Wykres porównujący algorytmy w przypadku losowych danych dla typu INT



Rysunek 22: Wykres porównujący algorytmy w przypadku losowych danych dla typu FLOAT

6 Podsumowanie i wnioski

Przeprowadzone eksperymenty w pełni potwierdziły teoretyczne założenia dotyczące złożoności obliczeniowej badanych algorytmów. Szczegółowa analiza wyników pozwala na sformułowanie następujących wniosków inżynierskich:

1. **Zjawisko najgorszego przypadku w Quick Sort (Stack Overflow):** Jak wynika z pomiarów, klasyczny Quick Sort z lewym lub prawym elementem osiowym (pivotem) jest całkowicie nieodporny na dane już posortowane (rosnąco, malejąco, a nawet w 33% i 66%). W tych przypadkach złożoność degradowała do $O(N^2)$, co w połączeniu z rekurencją doprowadziło do przepełnienia stosu wywołań już przy niewielkich rozmiarach tablic. Problem ten w ogóle nie wystąpił dla pivota środkowego, który idealnie dzielił ułożone zbiory, wykazując się bezbłędną stabilnością i czasem wykonania na poziomie optymalnym ($O(N \log N)$).

2. **Narzut operacyjny pivota losowego:** Mimo iż losowy wybór pivota zabezpiecza algorytm przed złośliwymi i specyficznymi rozkładami danych (brak błędów Stack Overflow), to w przypadku danych całkowicie losowych wariant ten okazał się najwolniejszy ze wszystkich implementacji Quick Sorta. Różnica ta wynika z kosztu cykli procesora potrzebnych na wywołanie funkcji pseudo-losowej `rand()` oraz operacji modulo przy każdym zejściu rekurencyjnym.

3. **Triumf podejścia hybrydowego (Hybrid Sort):** Połączenie algorytmu Quick Sort z Insertion Sort przyniosło zamierzony efekt optymalizacyjny. Algorytm Hybrid Sort osiągnął najlepsze sumaryczne czasy ze wszystkich badanych wariantów. Zaobserwowano ponadto, że optymalny próg odcięcia (*threshold*) to 25 oraz 100. Próg równy 10 generował nieznacznie gorsze czasy, co wskazuje na zbyt duże obciążenie mechanizmem rekurencji przy zbyt małych porcjach danych przekazywanych do sortowania przez wstawianie. Sukces hybrydy na tej konkretnej platformie testowej można również przypisać potężnej pamięci L2 i L3 procesora i5-13600KF, która doskonale buforowała małe, lokalne podtablice poddawane Insertion Sortowi.

4. **Przewaga sekwencji Franka i Lazarusa w Shell Sort:** Dla mniejszych tablic (do około 250 000 elementów) klasyczna sekwencja odstępów Shella utrzymywała konkurencyjny czas pracy. Jednakże przy skalowaniu do 5 000 000 elementów ujawniła się słabość naiwnego podziału przestrzeni – klasyczny Shell Sort (np. dla `float` - losowe) potrzebował 938 ms, podczas gdy zastosowanie złożonej sekwencji Franka-Lazarusa skróciło ten czas do 829 ms, wyraźnie ograniczając kwadratowy charakter wzrostu.

5. **Heap Sort a architektura sprzętowa:** Heap Sort, choć bezpieczny z uwagi na deterministyczną złożoność pesymistyczną $O(N \log N)$ i zużycie pamięci $O(1)$, okazał się wolniejszy od wariantów Quick i Hybrid (np. 576 ms vs 243 ms dla 5M `float` losowych). Zjawisko to jest zgodne z przewidywaniami i wynika z częstego „pudłowania" pamięci podręcznej procesora (ang. *cache misses*) podczas skakania po indeksach kopca, co łamie zasadę lokalności przestrzennej.

6. **Wpływ typu danych (`int` vs `float`):** Typ danych nie wpłynął na ogólny kształt krzywych złożoności, jednakże operacje na liczbach zmiennoprzecinkowych (`float`) systematycznie wymagały dłuższego czasu wykonania we wszystkich algorytmach o ok. 10-15%. Wynika to z wyższego narzutu operacyjnego koprocesora matematycznego (FPU) podczas porównywania ułamków w stosunku do prostych instrukcji arytmetycznych na liczbach całkowitych.

7 Literatura

[1] Thomas H. Cormen i in. *Wprowadzenie do algorytmów*. Warszawa: Wydawnictwo Naukowe PWN, 2012.